

What does it mean to be 'Turing complete'?

by Nathan Jones



Mindstorms Engineering, LLC
Embedded Systems Design & Education

Introduction

It's possible that you've heard the term "Turing complete" before, probably in reference to a computer or a programming language. Certain programming languages are or are not "Turing complete" and occasionally odd stuff, like [Conway's Game of Life](#), is called "Turing complete". But what, exactly, does this term mean and why is it important in the realm of computer science? To answer that, we'll travel back in time to the year 1936, before the first computer had ever been built.

Wait! Stop! You're going back in tiiiiime!!

The term "Turing complete" relates to famed mathematician Alan Turing, but to understand it's significance it's important to understand the nature of computation, to imagine a world where computers hadn't really yet been invented. Consider this: it is the early 1930s and modern technological advancements have made heretofore undreamed of machines possible. Although the world has struggled dutifully for thousands of years with simple mathematical aids such as abacuses, slide rules, and mechanical calculators, there are visionaries around the world who are not just imagining a place in which a lifeless device can perform advanced mathematical calculations but proving that such devices are capable of being built and then bringing them to life. The question they are trying to answer could be put, simply, as "What are the capabilities and limitations (theoretical or practical) of machines that perform mathematical operations?" Similar questions are still being asked today of computers that use photonic circuits or are built using quantum mechanics. The field of study that arose in the twentieth century around this question is called "computational theory". A brief discussion of computational theory will help set the stage for our main character, Alan Turing, and his amazing Turing machines.

Computational Theory 101

If you've ever heard the terms "Big O notation" or "NP-complete" then you're already at least a little bit familiar with computational theory. (Those terms are part of a branch of computational theory called "computational complexity" that seeks to understand whether or not a problem can be solved by a computer and also how long it would take a computer to solve a "solvable" problem.) Computational theory deals with machines performing mathematical operations that can be described as "functions on the natural numbers": $f: \mathbb{N}^k \rightarrow \mathbb{N}$. Let's dissect

that: the natural numbers are the positive integers: 0, 1, 2, 3, etc. And "function" here is synonymous with "algorithm" (more or less). So what we're saying is that computational theory studies those machines that can take one or more inputs in the form of a positive integer, follow a series of steps, and produce an output (also in the form of a positive integer). Integer addition is an exceedingly simple example of a "function on the natural numbers". But so is the U.S. tax form: after taking in a set of positive integers (your gross income, your deductions, etc), a series of steps can be followed that eventually inform you how much you owe the U.S. government (or how much the government owes you).

In fact, *every* computer program you've *ever* written can be described as a function on the natural numbers, since I can guarantee that all of them:

1. took in one or more positive integers (in the form of a binary value in memory or in a register that corresponded to a peripheral device like an ADC conversion result or UART receive byte),
2. followed a series of steps, and then
3. produced one or more positive integers (in the form of a binary value written to memory or to a register that corresponded to a GPIO port or SPI transmit register).

"But what about negative values or real numbers?", you say? Don't worry, there's a way to encode negative numbers as positive integers: it's called [two's complement](#)! There's a way to encode a real number as a positive integer too; it's called [IEEE 754](#) or floating-point format. (There are, of course, several ways to do either, but those are the most popular.) I'm sure that you're also already familiar with a multitude of ways to encode text (ASCII, UNICODE, etc), images (JPEG, PNG, etc), and music (MP3, AAC, WAV, etc) using positive integers. What this means is that the lowly set of "functions on the natural numbers" is infinitely large, that it describes *every* computer program you could ever think to write, and, with the proper encoding scheme, can also include algorithms that process negative numbers, real numbers, sound, music, or whatever else you can think of.

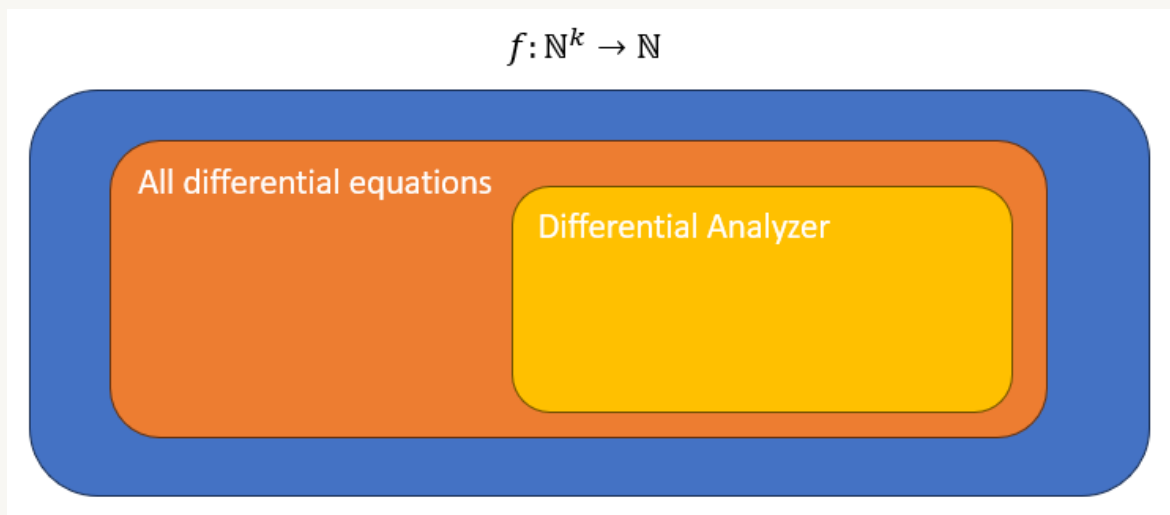
Enter Alan Turing

Arguably the most powerful computing device in operation at the time Alan Turing began thinking about computational theory was the "Differential Analyzer", designed and built by Vannevar (van-NEE-var) Bush in 1930 at MIT. The Differential Analyzer was a mechanical, analog calculator that was capable of solving certain classes of differential equations.

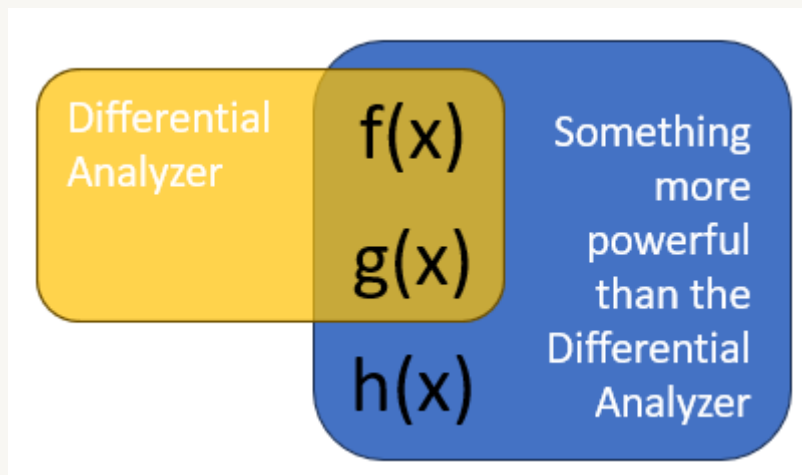


Vannevar Bush with his Differential Analyzer [\[1\]](#)

A differential equation could be thought of as one of the "functions on the natural numbers", so the Differential Analyzer could be said to have been able to solve a "subset of a subset of the functions on the natural numbers".



We can say it was the most "powerful" computing device in the world at the time of its construction because the subset of functions it could perform included anything that any other computing device could do. A more powerful computing device would have to be able to perform all of the functions on the natural numbers that the Differential Analyzer could do, plus more. Proving this is as simple/complicated as showing that the new machine can emulate the mechanics of the Differential Analyzer; a machine that can perform the same *mechanical* operations as the Differential Analyzer can perform the same *mathematical* operations, too.



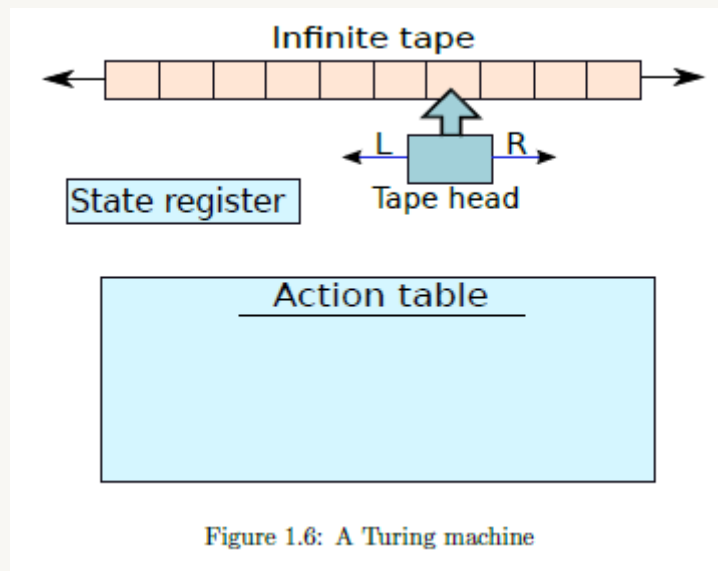
If the Differential Analyzer can perform functions $f(x)$ and $g(x)$ and another device can perform those functions *plus* $h(x)$, then the other machine is a more powerful computational device.

What, then, would it take to build a machine that could solve *all* differential equations? What would it take to solve *all mathematical functions*? Is that even a thing? Is there a function that a machine *couldn't* solve? These were the questions plaguing computational theorists like Alan Turing in the early twentieth century and which he sought to answer in his seminal paper titled "On Computable Numbers"[\[2\]](#), published in 1937.



Alan Turing (1912-1954) [\[3\]](#)

In "On Computable Numbers", Turing imagines an as-yet-unbuilt machine that he calls an "automatic machine" or "a-machine", which we have come to call a "Turing machine" (in his honor).

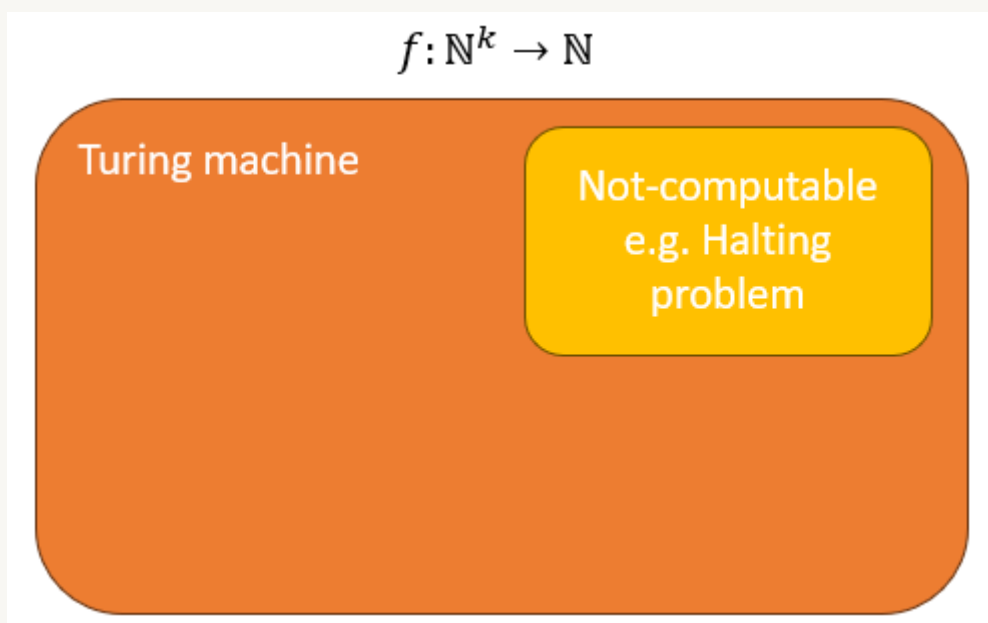


A block diagram of a Turing machine [4]

The Turing machine is deceptively simple: it consists solely of (1) a long strip of magnetic tape separated into squares upon which discrete symbols can be read and written to by a tape head and (2) a state machine that controls the behavior of the tape head. Given (1) the machine's current state and (2) the symbol underneath the tape head, a Turing machine can do one or more of only three things:

- write a new symbol onto the tape,
- move the tape head left or right, and
- transition to a different state.

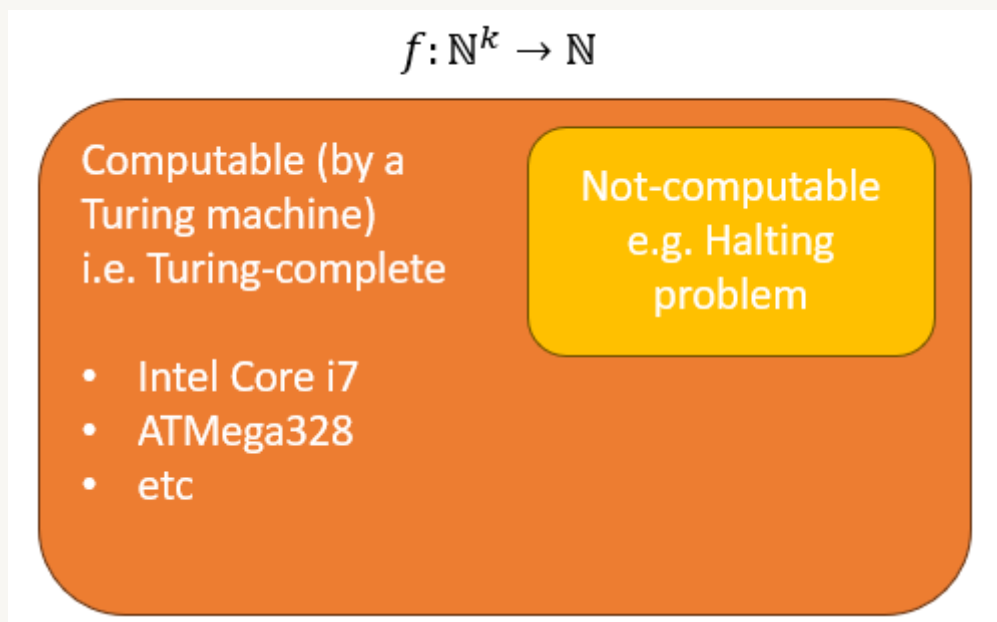
Turing uses this machine to prove that there *are* some problems that can't be solved by computers (such as the [Halting Problem](#), which he proves in his paper, though there are others) and that a single machine, the Turing machine, is capable of solving *all other functions on the natural numbers* if it's given enough time and memory.



A Turing machine is capable of computing all computable problems.

(Do you find that a bit hard to believe? [Here's](#) a program that you can run on a [simulated Turing machine](#) which adds together two binary numbers of arbitrary length. Think about that: the Turing machine has *no arithmetic hardware* and yet here it is, performing multi-bit addition.)

Let that sink in for a second: a simple tape head and finite state machine is capable of executing *every* computer program you've ever seen or even dreamed of (at least, the solvable ones). Turing isn't actually able to *prove* this statement, but no one's been able to disprove it either in the nearly 100 years that it's been around. Furthermore, any computational device that can *mechanically* emulate a Turing machine is called "Turing complete" and can be said to have the same quality. The ATmega328 microcontroller on an Arduino Uno R3 development board, being Turing complete, can compute *exactly* the same things that an Intel Core i7 can, it might just take it a lot longer. Heck, an ATmega1284 (a larger cousin to the ATmega328, the microcontroller on the Arduino Uno R3 boards) can even [run Linux](#). [Edited 11 Apr 24; thanks for the correction, @fjrg76!]



Every machine that can emulate a Turing machine is "Turing complete" and can perform *exactly* the same operations as any other computer that is Turing complete (if it is given enough time and memory).

This also means that things that aren't even electronic circuits can be "Turing complete". A programming language can be called Turing complete if it's possible to use the programming language to emulate a Turing machine. Doing so would prove that it's possible to use that programming language to implement all solvable problems. Typical programming languages, such as C, C++, Java, and Python, are, of course, Turing complete, but then so are things that aren't strictly considered to be programming languages like [make](#), [Notepad++](#), and the [x86 MOV instruction](#). Additionally, it's possible to build Turing complete devices (i.e. computers) out of anything that can emulate a Turing machine, such as [marbles](#), [water valves](#), and [Legos](#).

What does it mean?

Some good, some bad. Under the "neat/good/useful" umbrella is the idea that whatever processor you're writing code for, using whatever programming language you're using, you can write a program to do *anything*, any other processor can do, as long as you're willing to wait and your processor has enough memory. Perhaps this is only marginally useful, since processor cycles and memory space are fairly plentiful these days; running Linux on an ATmega1284 is definitely more of a novelty than anything else. But if you ever find yourself limited by your hardware or your software, then perhaps you can use this fact to get your project over the finish line. Are you running out of peripherals? Perhaps you can bit-bang a communication protocol. Are you running out of program space? Perhaps you can write a small program to emulate a simple processor and then interpret instructions that you read from some piece of external memory. Or perhaps your emulated processor could even extend the memory limits of your ISA, so that you aren't limited by a 12-bit data memory space. The possibilities are nearly endless.

Additionally, it means that there are some neat processor architectures out there which one could implement on an FPGA that might actually be more efficient than other, more common types of soft cores. For instance, routing the signals for a 64-bit processor is cumbersome and wastes space inside an FPGA. An alternative design is to use a 1-bit ALU and instead of processing 64 bits at a time (on a single clock cycle), processing 1 bit at a time over 64 clock cycles. This is called a "[bit serial](#)" processor and many of the earliest computers implemented this type of processor (out of necessity; a 64-bit ALU built from vacuum tubes would be huge!). This might seem like a set-back until you recognize that since routing a 1-bit signal is MUCH easier than routing a 64-bit signal, 64 bit-serial processors (called "massively parallel processors") can take up *less space* than an equivalent 64-bit processor. Less silicon means less power or, if you're inclined to turn your dials up to 11 and chose to take up the space saved with *more bit-serial processors*, then this massively-parallel design could allow for more processing power than a normal processor using the same silicon area. [SERV](#) is a popular bit-serial implementation of the RISC-V ISA.

Under the "bad/scary/terrifying" umbrella is the idea that a malicious actor, who's allowed access to a part of your system from which they can create a Turing machine, can effectively create a virtual machine inside your system and run their own arbitrary programs. These "weird machines" pose a significant security threat. In fact, this is exactly how NSO accomplished their [zero-click exploit of iMessage](#): by creating a Turing-complete machine out of an image compression program called JBIG2 that allowed them to access the entire memory space of the affected iPhone.

Using over 70,000 segment commands defining logical bit operations, they define a small computer architecture with features such as registers and a full 64-bit adder and comparator which they use to search memory and perform arithmetic operations. It's not as fast as Javascript, but it's fundamentally computationally equivalent. [\[6\]](#)

I want to emphasize here the "zero-click" part of "zero-click exploit": users weren't required to click a malicious link for the exploit to work, it happened *even when they took no action*. "Short of not using a device, there is no way to prevent exploitation by a zero-click exploit; it's a weapon against which there is no defense," states Project Zero, which is probably why they described this attack as "one of the most technically sophisticated exploits we've ever seen." Gwern Branwen puts it this way:

Any sort of scripting interface or API, even if locked down, may not be locked down quite enough...if one is clever, it provides an escape hatch from a system which is small, predictable, controllable, and secure, to one which could do anything. (And will once an attacker has something to say about it.) [7]

Blessedly, the exploitation of iMessage has since been thwarted. Project Zero reported in the above article that "The vulnerability discussed in this blog post was fixed on September 13, 2021 in [iOS 14.8](#) as CVE-2021-30860."

Wrapping Up

The idea of "Turing completeness" may be mathematical and difficult to grasp, but the implications are quite large. In the end, I hope you understand a little more about the computational theory that describes how computers work and that you're newly aware of all that you can actually do with the things you program. Happy hacking!

Footnotes

[1] William Morton Pottenger and D. Hemmendinger, "Computer - History of computing," Encyclopædia Britannica. Jan. 30, 2019. Available: <https://www.britannica.com/technology/computer/History-of-computing>

[2] A. M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem," Proceedings of the London Mathematical Society, vol. s2-42, no. 1, pp. 230–265, 1937, doi: <https://doi.org/10.1112/plms/s2-42.1.230>.

[3] Biography.com Editors, "Alan Turing - Education, Movie & Quotes," Biography, Jul. 22, 2020. <https://www.biography.com/scientists/alan-turing>

[4] S. R. Sarangi, *Basic Computer Architecture*. 2021.

[5] Turing machine program to add together two arbitrarily-long binary numbers. The two numbers appear on the line that starts "tape", with each digit separated by a space and each number terminated by a dash (-). The first digit of the first number should be wrapped in a pair of brackets ([]) to indicate that the Turing Machine should start with it's tape head over that spot.

```
states s0 s1 s2 s3 s4 s5
symbols 0 1 -
```

```
tape add12and15 [1] 1 0 0 - 1 1 1 1 -  
result add12and15 1 1 0 1 1 - - - - - [-]  
action s0 0 s0 0 l  
action s0 1 s0 1 l  
action s0 - s1 - l  
action s1 0 s1 0 l  
action s1 1 s1 1 l  
action s1 - s2 - r  
action s2 0 s2 1 r  
action s2 1 s3 0 r  
action s2 - s5 - l  
action s3 0 s3 0 r  
action s3 1 s3 1 r  
action s3 - s4 - r  
action s4 0 s0 1 l  
action s4 1 s4 0 r  
action s4 - s0 1 l  
action s5 1 s5 - l  
action s5 - *halt* - l
```

[6] “Project Zero: A deep dive into an NSO zero-click iMessage exploit: Remote Code Execution,” Project Zero, Dec. 15, 2021. <https://googleprojectzero.blogspot.com/2021/12/a-d...>

[7] G. Branwen, “Surprisingly Turing-Complete,” gwern.net, Dec. 2012, Accessed: Oct. 06, 2023. [Online]. Available: <https://gwern.net/turing-complete>